

VOYAGEWRIGHT

PROJECT HELM

The Captain Operations and Participating-Captain System

Governing Architecture and Product Requirements Document

GOVERNING PRINCIPLE
Captain authority and Player participation are independent dimensions. The same person may operate a Voyage and participate in it as a normal Player, but Captain privilege must never contaminate Player-safe projections, personal history, artifacts, or story visibility.

Version 1.0 | August 8, 2026

Repository baseline reviewed: Kgray44/treasurehuntSoT main at f1c2f22dd935322c1a71eb80c51592f243dc196d

Document Control

Field	Value
Program	Project Helm
Subsystem	Captain Operations and Participating-Captain System
Document type	Governing architecture and product requirements
Version	1.0
Status	Governing baseline
Prepared	August 8, 2026
Repository baseline	Kgray44/treasurehuntSoT main at f1c2f22dd935322c1a71eb80c51592f243dc196d
Current capability being deepened	FT-007 - Captain Voyage Operations
Primary surfaces	Captain Library, Captain Voyage Console, Voyage setup/preflight, Player perspective switch, recovery surfaces
Core implementation rule	Helm consumes canonical One Voyage truth; it must not create another progression engine or leak Captain-only knowledge into Player experiences.
Development rule	Every phase must satisfy the Continuous Development and Mainline Integration Standard and merge independently into main.

Contents

1. Executive Summary
2. Project Identity and Product Vision
3. Current Repository Context
4. Core Product Outcomes
5. Scope and Non-Goals
6. Non-Negotiable Design Principles
7. Canonical Architecture and Ownership
8. Cross-Project Boundaries
9. Captain Authority and Player Participation Model
10. Captain Participation Modes
11. Identity, Membership, and Historical Invariants
12. Captain Library Evolution
13. Helm Dashboard Information Architecture
14. Voyage Operational Status Model
15. Crew Operational Projection
16. Presence, Synchronization, and Readiness
17. Needs Attention and Warning System
18. Live Progress Map
19. Event Stream and Operational History
20. Captain Command Model
21. Preview, Stage, Confirm, Execute
22. Hints and Player Assistance
23. Approvals, Verification, and Manual Fallback
24. Player Perspective and Safe Preview
25. Voyage Preflight and Launch Readiness
26. Launch, Scheduling, and Waiting Room Coordination

27. Realtime, Concurrency, and Multi-Device Operation
28. Recovery and Reconciliation
29. Participant Lifecycle Management
30. Emergency Controls and Degraded Operation
31. Captain Intelligence and Privacy Boundaries
32. Security and Authorization
33. Audit, Diagnostics, and Support Boundaries
34. Data Model and Service Contracts
35. API and Projection Contracts
36. Responsive Captain Experience
37. Accessibility
38. Lanternwake Motion and Presentation
39. Integration with Drydock, Landfall, Watchglass, Wayfarer, Wakebook, and Admiralty
40. Performance and Operational Architecture
41. Testing and Acceptance Matrix
42. Implementation Phases Overview
43. Phase 1 - Take the Helm
44. Phase 2 - Read the Deck
45. Phase 3 - Give the Orders
46. Phase 4 - Weather the Passage
47. Phase 5 - Clear for Voyage
48. Mainline Integration Architecture
49. Risks and Failure Modes
50. Final Acceptance Criteria
51. Governance and Change Control
52. Glossary
53. References

Appendix A. Captain Command Catalog

Appendix B. Operational Projection Contract

Appendix C. Privacy Matrix

Appendix D. Mainline Safety Matrix

Appendix E. Acceptance Checklist

1. Executive Summary

Core Product Outcome
A Captain should be able to prepare, launch, operate, participate in, recover, and complete a Voyage from one truthful command environment without being excluded from the adventure or exposed to unsafe omniscience.

Project Helm transforms the existing Captain capability from a useful but comparatively thin control surface into a complete live-Voyage operating system. Voyagewright already has mainline Captain Voyage operations: creation, crew readiness, idempotent commands, pause/resume, and an operational audit trail. Helm does not replace those capabilities. It turns them into a coherent, deep, privacy-safe command experience.

The most important architectural change is the separation of authority from participation. A person may have Captain authority over a Voyage and simultaneously hold a normal Player membership in that same Voyage. The Captain side of that person may issue authorized operational commands; the Player side receives only Player-safe Chronicle content, participates in choices, earns artifacts according to ordinary recipient rules, and enters personal history exactly like any other Player.

Helm also expands the Captain experience around real operational needs: preflight, live crew state, synchronization health, pending requests, progress visibility, contextual commands, warnings, provider status, safe hints, recovery, participant management, and degraded-mode operations. It is designed for active use on a second monitor, laptop, tablet, or phone while the Voyage is actually happening.

Helm is not an authoring system and not a platform-administration system. Creator Studio remains the place where a Chronicle is built. Admiralty remains the privileged platform administration plane. One Voyage remains the sole progression authority. Helm is the human command layer over that truth.

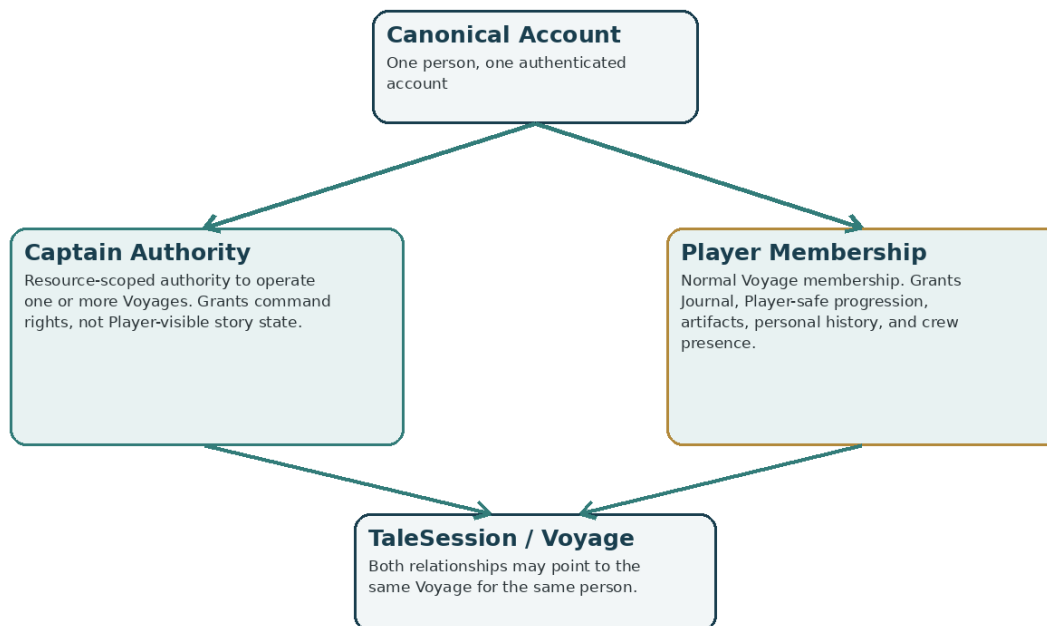
2. Project Identity and Product Vision

The project is named Project Helm. A helm is where a vessel is actively guided while underway. The name therefore fits the subsystem that guides a live Voyage without owning the authored Chronicle or the platform itself.

The formal subtitle is The Captain Operations and Participating-Captain System. The product vision is a Captain experience that feels like a serious command center while remaining approachable enough for a friend running a date-night Chronicle from a phone.

Helm should reduce the mental load of running a Voyage. Instead of making Captains infer state from scattered pages, it should surface what matters now, what needs attention, which actions are valid, what will happen if an action is taken, and how to recover safely when reality performs its usual sabotage.

Captain authority and Player participation are independent dimension



3. Current Repository Context

The current generated Feature Catalog records FT-007, Captain Voyage Operations, as MAINLINE. Its meaningful subfeatures are Voyage creation, crew readiness, command idempotency, pause and resume, and an operational audit trail. The primary surfaces are /captain and /captain/voyages/[id].

The current command domain already contains a substantial command vocabulary covering chapter preparation/release, solving/completion, hints, maps/routes, artifacts, side quests, journal/log entries, finale state, pause/resume, undo, and reconciliation. Commands carry expected sequence and idempotency evidence, and the command layer distinguishes committed progression events from staged actions.

The current implementation also contains presence evidence and synchronization description, Captain command previews, staged command records, audit events, and safe command conflict behavior. These are valuable foundations. The gap is not that Captain has no backend. The gap is that those capabilities are not yet assembled into the expansive, deeply informative operating experience envisioned for Helm.

The historical Tall Tale Platform specification also established Captain Library groups such as Needs Attention, Active Voyages, Ready to Launch, Invitations, Published Tales, Completed Playthroughs, and Drafts. It required active Voyage cards to show exact versions, Player-safe versus Captain-facing checkpoints, sync health, pending actions, safe Player preview, and valid state-dependent controls. Helm adopts and deepens that intent rather than inventing a competing Captain model.

Existing capability	Current strength	Helm extension
Captain Voyage Operations	Creation, launch, commands, pause/resume, audit	Unified operational projections and deep live command UX
Admin command domain	Broad typed command set with risk and idempotency	Contextual action rail, previews, confirmations, command availability and receipts
Presence evidence	Heartbeat, route, acknowledgment sequence, lag	Crew operational state, warnings, reconnect guidance, sync visualization
Player/Captain separation	Server-side role authorization and Player-safe projection	Captain+Player dual mode without privilege leakage
Invitation lifecycle	Individual invitation and crew membership foundation	Preflight, readiness, participant management, launch coordination
One Voyage progression	Canonical events and state	Helm consumes it; never duplicates progression logic

4. Core Product Outcomes

- A Captain can run a Voyage from a coherent command environment rather than scattered route-specific controls.
- A Captain can choose Captain-only or Captain+Player participation for each Voyage where policy permits.
- A participating Captain receives a normal Player membership and Player-safe experience while retaining separate Captain authority.
- Crew readiness, presence, sync health, pending requests, acknowledgment state, and last safe activity are visible without exposing unrelated private data.
- Context-sensitive Captain commands show only when meaningful, explain prerequisites, preview effects, require risk-appropriate confirmation, and execute through canonical One Voyage authority.
- Preflight identifies missing players, invalid version/readiness, unsupported providers, accessibility concerns, missing media, or other launch blockers before launch.
- Recovery surfaces guide safe reconciliation when devices disconnect, sessions restart, providers fail, or presentation acknowledgments are missing.
- Captain actions remain audited, idempotent, sequence-safe, and non-destructive under retries or multi-device use.
- Captain UI is usable from desktop, second monitor, tablet, and phone during gameplay.
- Completed Voyages preserve Captain participation and interventions for authorized historical views without rewriting Player history.

5. Scope and Non-Goals

In scope

- Captain-only and Captain+Player participation
- Captain Library and live Voyage Console
- Crew operational state and presence
- Needs Attention queue and warning taxonomy
- Progress graph and pending requirements
- Command preview/stage/execute UX
- Hints, approvals, verification fallback
- Preflight and launch readiness
- Recovery, reconciliation, reconnect and participant repair
- Captain history and intervention summaries
- Responsive/accessibility/motion completion
- Mainline-safe phased implementation

Explicit non-goals

- No second progression engine
- No Creator Studio authoring controls inside live Helm
- No platform-wide Administrator operations; Admiralty owns those
- No new public Community social system
- No replacement account system
- No raw database repair UI
- No arbitrary command scripting
- No exposure of private Player reflections, secrets, or unrestricted answer text
- No fake device telemetry when the backend cannot support it
- No automatic overriding of Chronicle rules merely because the Captain wants a convenient button

6. Non-Negotiable Design Principles

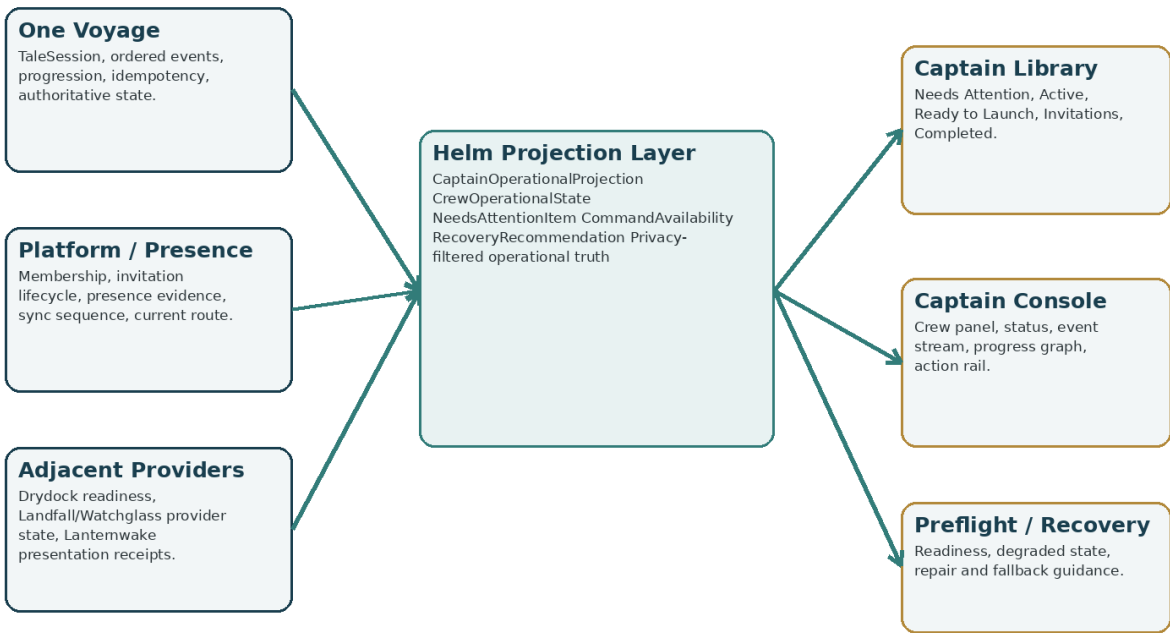
Principle	Requirement
Authority and participation are orthogonal	Captain authority never substitutes for Player membership, and Player membership never grants Captain commands.
Operational truth, not omniscience	Helm exposes what the Captain needs to operate the Voyage, not every private fact the database contains.
One Voyage remains authoritative	Every mutation routes through canonical progression, invitation, membership, verification, and audit services.
Context before controls	Actions appear with current prerequisites, consequences, and state validity; dangerous universal button walls are prohibited.
Recovery is a first-class workflow	Disconnections, stale state, provider failure, missed presentation, and participant loss have governed repair paths.
State must be truthful	Pending, committed, delivered, presented, and acknowledged are not collapsed into one optimistic success state.
Captain should be able to play	Creating or operating a Voyage must not automatically exile the Captain from Player participation.
No authoring during live operation	Creator and Captain workspaces may be linked, but live operation must not be surrounded by draft editing controls.
Mobile is operational, not decorative	Critical Captain tasks remain usable on a phone with large targets, low navigation depth, and explicit confirmation.
Every phase lands cleanly	Project Helm follows the Continuous Development and Mainline Integration Standard phase by phase.

7. Canonical Architecture and Ownership

Helm is a projection-and-command subsystem. It reads canonical Voyage truth, builds purpose-limited Captain projections, and submits mutations through existing authoritative services. The user interface never becomes an authority simply because it has a dramatic brass button.

The architecture must distinguish source truth, operational projection, command intent, command receipt, presentation state, and historical projection. Each layer has a different security and consistency contract.

Helm consumes canonical truth through explicit operational projections



Layer	Authority	Helm responsibility
Account and capability	Wayfarer/Homeport	Resolve authenticated person and Captain capability
Voyage lifecycle and progression	One Voyage	Read projections; submit authorized commands
Membership and invitations	One Voyage / Platform	Read crew state; request authorized participant lifecycle changes
Presence and event delivery	Platform realtime	Summarize sync health; never fabricate presence
Presentation	Lanternwake	Display receipts and replay state; do not invent success
Personal history	Wayfarer / Wakebook	Provide Captain participation/intervention source facts only
Platform admin	Admiralty	Expose no global admin powers inside Helm

8. Cross-Project Boundaries

Adjacent project	Helm may consume	Helm must not own
One Voyage	Session state, command services, events, memberships, verification results	Progression rules, event authority, runtime variables
Wayfarer/Homeport	Account identity, capabilities, profile-safe display	Authentication, account lifecycle, personal history truth
Wakebook	Historical display contracts later	Another personal history store
Drydock	Chronicle readiness and validation receipts	Static analysis or simulation engine
Landfall	Location/provider state and manual-fallback contracts	Geospatial truth or location collection
Watchglass	Verification request/evidence status	Vision model or certification
Lanternwake	Captain scene contracts, receipts, reduced motion	Animation authority
Shipwright	Creator-side operational metadata later	Authoring UI or Story Block schema
Admiralty	Escalation/support link and platform health summaries later	Admin permissions, server settings, account inspection
Tideglass	Edition comparison links in historical/launch views later	Edition diff engine

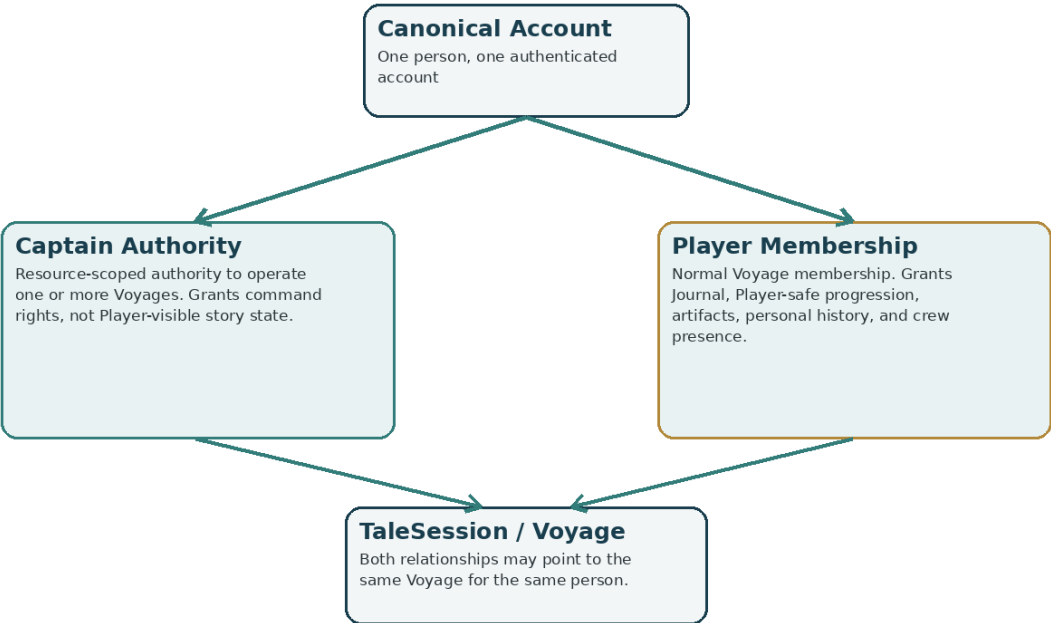
9. Captain Authority and Player Participation Model

Captain authority answers: may this person operate this Voyage? Player membership answers: is this person participating in this Voyage as a Player? These questions must be modeled independently.

A Captain+Player has both relationships. Their Captain requests use the Captain-authorized server path. Their Player pages use the same Player-safe projection as any other member. The server must never infer that because the authenticated account has Captain capability, Player APIs may return Captain-only fields.

Where the current data model can represent both dimensions through scoped role assignments plus normal PlaythroughMembership, prefer that existing structure. Introduce a new assignment table only if current relations cannot express revocation, scope, history, and multiple Captains safely.

Captain authority and Player participation are independent dimension



Authority	Player membership	Expected behavior
No	No	No access except public/deep-link flows
No	Yes	Normal Player experience only
Yes	No	Captain-only operator; no personal Player progression
Yes	Yes	Participating Captain: full normal Player membership plus separate Captain console

10. Captain Participation Modes

Captain Only

Captain operates the Voyage without receiving Player membership. This remains useful for dedicated facilitators, test operators, or Chronicles where the Captain must remain outside the narrative.

- No Player Journal
- No personal artifact grants
- No Player choices
- No personal Voyage completion record unless a separate Captain history record is governed
- Captain Console remains available

Captain + Player

The Captain is also added as an ordinary Player membership. They receive every Player entitlement and restriction applicable to that membership.

- Player-safe Journal only
- Normal crew identity
- Normal artifact recipient evaluation
- Normal choice participation
- Normal personal history and Keepsake projection
- Captain commands available only through Captain workspace/overlay

Mode changes

Changing between Captain-only and Captain+Player must be explicit, audited, and constrained by Voyage state. Adding Player membership after launch must follow ordinary late-join policy. Removing a participating Captain as Player must not silently revoke their Captain authority, and revoking Captain authority must not silently remove their Player membership.

11. Identity, Membership, and Historical Invariants

- One account may be assigned Captain authority to multiple Voyages without becoming a Player in all of them.
- A participating Captain has exactly one normal Player membership per Voyage, subject to the same uniqueness/idempotency rules as other Players.
- Captain authority revocation and Player membership removal are separate events with separate audit records.
- Artifact grant recipient evaluation uses actual event-time Player membership, not Captain capability.
- Personal history records derive from Player membership and canonical events, not from the fact that an account operated the Voyage.
- Captain intervention history may be retained in authorized operational/history projections without altering Player history.
- Completed Player records remain pinned to the exact published edition even if the same person later runs another edition as Captain.
- Historical display names and roles use governed snapshots; present-day profile edits do not rewrite prior Voyage facts.

12. Captain Library Evolution

The Captain Library remains the organizational center for operational work. Helm deepens it into a prioritized command queue rather than turning it into an authoring dashboard.

The default view should answer three questions immediately: What needs me now? What is currently underway? What can safely launch next? Secondary sections preserve invitations, published Chronicles, completed runs, and optional Studio shortcuts.

Group	Purpose	Key signals
Needs Attention	Voyages requiring Captain intervention	Waiting Player, pending verification, connection loss, stale sequence, blocked provider, unresolved warning
Active Voyages	Currently launched sessions	Current checkpoint, sync health, pending action, pause state, recent events
Ready to Launch	Configured but not launched	Crew readiness, edition, validation, provider readiness, schedule
Invitations	Invitation lifecycle management	Viewed/accepted/expired/revoked/replaced, recipient, delivery state
Completed	Historical operations	Crew, edition, completion, intervention summary, issues/recovery used
Published Chronicles	Start new Voyage	Latest eligible edition, readiness, active run count
Creator shortcut	Optional navigation only	Draft health or validation summary; never embedded authoring controls

13. Helm Dashboard Information Architecture

The active Voyage dashboard should use a stable layout so a Captain learns where to look under pressure. The same conceptual regions should survive desktop, tablet, and phone adaptations.

The information hierarchy should prioritize live status and crew state above historical detail. Rare administrative actions belong behind deliberate menus or confirmation surfaces.

Region	Primary content
Voyage Header	Chronicle, edition, Voyage name, lifecycle, elapsed time, connection state
Needs Attention Rail	Urgent warnings and pending actions sorted by severity/time
Crew Panel	Membership, presence, readiness, sync lag, pending request, safe current state
Progress Map	Current block/chapter, branch state, side quests, requirements, finale readiness
Action Rail	Context-valid Captain commands with risk and prerequisites
Event Stream	Ordered canonical operational events and Captain interventions
Recovery Drawer	Reconcile, reconnect, fallback, participant repair, diagnostics
Perspective Control	Switch to Player view for participating Captain or safe nonmutating preview

14. Voyage Operational Status Model

Helm must derive a Captain-facing operational status from canonical session, membership, presence, provider, and presentation facts. The status is a projection, not another state machine that can diverge from the Voyage.

Operational status can be richer than the underlying lifecycle but must always identify its source facts and last synchronization time.

Operational state	Meaning	Typical Captain response
SETUP	Voyage exists but setup incomplete	Complete configuration/invitations
WAITING_FOR_CREW	At least one required participant not ready	Inspect crew readiness
READY	Launch prerequisites satisfied	Launch when intended
ACTIVE_HEALTHY	Voyage active and synchronized	Observe/operate normally
ACTIVE_ATTENTION	Active but pending request/warning exists	Open Needs Attention
PAUSED	Authoritative pause state	Resolve reason and resume when safe
DEGRADED	Provider/realtime/presentation issue with safe continuation	Use fallback/recovery path
RECONCILING	State mismatch or restart recovery underway	Wait/inspect reconciliation receipt
COMPLETED	Terminal progression state	Review summary/history
ABANDONED/CANCELLED	Explicit terminal non-completion	Review reason, restart as new Voyage if needed

15. Crew Operational Projection

Helm must not display raw Player models. It requires a purpose-built CaptainCrewMemberProjection that contains only operationally justified information.

The projection should answer whether a participant is expected, accepted, ready, present, synchronized, waiting on the Captain, blocked, or removed. It should avoid sensitive account/security information and private story data.

Field family	Examples	Privacy rule
Identity	Display name, avatar where permitted, crew role	Use profile-safe projection only
Membership	Invited, accepted, ready, active, removed, completed	Canonical membership state
Presence	Connected, recently lost, stale, last seen, active device count	No raw IP/device fingerprint
Synchronization	Acknowledged sequence, lag, last server sync	Operational only
Current activity	Player-safe current section/checkpoint, pending action	No future or private content
Request state	Hint request, approval waiting, verification waiting	Show type and status; sensitive payload only if policy permits
Accessibility readiness	Required accommodation flags relevant to launch	Show only operational preference, never unrelated health/private data

16. Presence, Synchronization, and Readiness

Current code already models heartbeat evidence, disconnection, acknowledged sequence, route, active device count, lag, and synchronization. Helm should present those facts clearly without implying more certainty than exists.

A green Connected indicator must mean a recent authenticated heartbeat, not that the Captain saw a browser tab once. A synchronized indicator should require active presence and zero acknowledged event lag according to the current provider contract.

Readiness is broader than presence. A participant can be connected but not ready because an invitation is incomplete, required media is unavailable, permissions are missing, or a governed preflight check failed.

Signal	Captain-facing wording	Never imply
Fresh heartbeat + no lag	Connected and synchronized	That the Player is looking at a specific clue unless route/presentation evidence says so
Fresh heartbeat + lag	Connected - catching up	That events were presented or understood
Recent loss	Connection recently lost	That the Player intentionally left
Stale evidence	Not currently connected	That the device is off or account is invalid
Acknowledgment missing	Reveal not yet acknowledged	That presentation definitely failed
No presence data	Presence unknown	Disconnected

17. Needs Attention and Warning System

Needs Attention is not a generic notification feed. It is a prioritized operational queue for conditions where Captain action may materially help the Voyage.

Warnings should be deduplicated, source-bound, severity-rated, and dismissible only when dismissal is safe. Persistent warnings reappear when the underlying condition remains.

Severity	Examples	Behavior
INFO	Player joined, nonblocking provider degraded	Visible in event stream; optional dashboard note
NOTICE	Player requested hint, replay requested	Needs Attention item; no alarm styling
WARNING	Player disconnected, presentation acknowledgment overdue, provider fallback recommended	Prominent queue item with suggested action
HIGH	Stale command sequence, required Player blocked, manual verification needed	Action required before dependent command
CRITICAL	State integrity conflict, unsafe provider result, impossible recovery without protected escalation	Block unsafe actions and require reconciliation/admin path

18. Live Progress Map

The progress map is a Captain-facing operational projection of the Chronicle graph and current Voyage state. It does not replace Creator Studio and must not permit editing the graph.

It should distinguish completed, active, upcoming-safe, optional, blocked, hidden-by-policy, and finale requirement nodes. Captain-only operational annotations may be shown where the published edition explicitly contains them.

For participating Captains, opening the Player perspective must use the ordinary Player projection. The Captain progress map may reveal more operational structure, but those details must never bleed into the Player Journal or Player-side network payloads.

Visual state	Meaning
Completed	Authoritative completion event exists
Current	Canonical current block/chapter or active requirement
Pending Captain	Progression awaits Captain-authorized action
Pending Player	Progression awaits allowed Player action
Provider waiting	Verification/location/vision provider has not resolved
Optional	Side quest or branch not required for main completion
Blocked	Prerequisite not currently satisfiable
Unknown/Protected	Details intentionally not available to this projection

19. Event Stream and Operational History

Helm should show a concise live event stream built from canonical ordered Voyage events, invitation/membership events, presentation acknowledgments, presence transitions, and Captain audit events. The stream is a projection over existing event authorities, not a new event log.

Each event should have a stable type, timestamp, actor-safe label, sequence where applicable, correlation ID where useful, and bounded safe metadata. Sensitive story payloads, answer text, raw invitation secrets, or private reflections are never logged merely to make the timeline look detailed.

Filters should include crew, progression, hints, verification, artifacts, connection, Captain actions, recovery, and system. Search must use safe labels only.

20. Captain Command Model

Helm inherits the canonical command vocabulary and command risk model. It should rationalize presentation around intent rather than exposing a flat list of internal command identifiers.

Existing command families cover progression, hints, map/route reveal, artifacts, side quests, journal/log entries, finale, pause/resume, undo, and reconciliation. Helm should group them contextually and derive availability from current session state, target state, policy, and Captain capability.

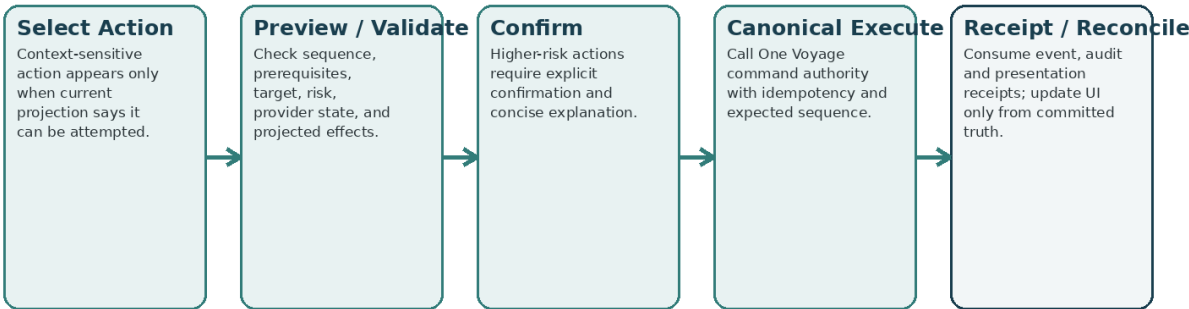
Intent group	Representative canonical commands	UI treatment
Progression	PREPARE_CHAPTER, RELEASE_CHAPTER, MARK_SOLVED, COMPLETE_CHAPTER	Chapter/status panel
Hints	PREPARE_HINT, RELEASE_HINT, RELEASE_NEXT_HINT	Player request or hint drawer
World/Map	REVEAL_MAP, REVEAL_ROUTE	Chart/progress context
Artifacts	AWARD_ARTIFACT, REVEAL_ARTIFACT_SILHOUETTE, CONNECT_ARTIFACTS	Artifact context
Side quests	DISCOVER_SIDE_QUEST, UPDATE/ADVANCE/COMPLETE_SIDE_QUEST	Echo/side-quest panel
Journal/Log	ADD_JOURNAL_ANNOTATION, ADD_LOG_ENTRY, RELEASE_JOURNAL_ENTRY	Narrative operation panel
Finale	TEASE_FINALE, UPDATE_FINALE_REQUIREMENT	Finale requirements
Lifecycle	PAUSE, RESUME	Voyage header
Recovery	UNDO_LAST, REQUEST_RECONCILIATION	Recovery drawer

21. Preview, Stage, Confirm, Execute

High-quality Captain operation requires a command lifecycle rather than instant button-to-mutation behavior. Existing backend support for preview and staging should be surfaced consistently.

Preview shows current sequence, projected sequence, prerequisites, affected systems, and whether execution is currently valid. Staging reserves an operational intent where supported. Confirmation communicates risk in human terms. Execution submits the idempotent command against the expected sequence. The final interface state comes from the authoritative receipt.

Captain command lifecycle



No UI animation, toast, or optimistic state may claim success before canonical execution is committed.

Risk	Default UI requirement
LOW	Direct action permitted when reversible/nonmutating; still show result truthfully
MEDIUM	Short preview or inline confirmation; clear target
HIGH	Explicit confirmation dialog with consequence, target, and stale-state handling
CRITICAL	Not exposed through ordinary Helm unless a separately governed emergency contract exists

22. Hints and Player Assistance

Helm should make assisting a Player easy without making the Captain manually inspect private data. A hint request can surface the Player, relevant Chronicle step, hints currently allowed, prior hint count, timing if governed, and any Captain-authored note intended for this operational context.

Where the Chronicle defines multiple hints, Helm should show only hints legitimately available to the Captain at the current point. Releasing a hint must be authoritative, auditable, and safe under retry.

The Captain should also be able to send a bounded operational note when the Chronicle supports it, but Helm is not a direct-messaging platform. Freeform notes must have clear visibility scope and retention rules.

- Hint availability derives from the pinned published edition, not the current draft.
- Player request status is separate from whether a hint has been released.
- A participating Captain must not see unreleased hints in Player view.
- Hint release receipts should identify delivery/presentation state separately from mutation success.

23. Approvals, Verification, and Manual Fallback

One Voyage already defines a narrow verification provider boundary and Captain override capability. Helm should make that operationally understandable without letting providers or Captain UI bypass progression rules.

For approvals, the Captain receives a summary of the verification request, the provider/evidence status, the governed decision options, and the consequence of approving or rejecting. Raw evidence is shown only when policy and privacy permit.

Manual fallback must be explicit and audited. It should identify why the automated provider could not confirm, whether manual approval is permitted by the Chronicle, and what progression event will result.

Case	Captain sees	Captain may do
Text/choice approval	Safe request state and allowed outcome	Approve/reject if block policy permits
Watchglass evidence	Certified result/uncertainty and safe evidence summary	Approve/reject/manual fallback under provider contract
Landfall arrival	Confidence/accuracy and fallback eligibility	Confirm manual arrival only when Chronicle allows
Provider unavailable	Provider state and safe fallback options	Choose governed fallback; never fabricate provider success
Stale evidence	Reason evidence is stale	Request new evidence or use allowed manual path

24. Player Perspective and Safe Preview

A participating Captain must be able to move between Player and Captain perspectives without creating two identities. Player View opens the ordinary Player route and ordinary Player-safe API projection for that membership.

A nonparticipating Captain may use Safe Preview as Player where supported. Preview is synthetic/nonmutating, visibly marked, and incapable of altering the real Player session. This preserves the historical platform requirement that Captain preview cannot mutate live progress.

Perspective switching must preserve focus, avoid accidental command execution, and make current mode obvious. The Player surface must never add Captain-only buttons simply because the account also has Captain authority.

25. Voyage Preflight and Launch Readiness

Helm preflight turns launch from a hopeful button press into a governed readiness decision. It gathers all launch-relevant facts that can be known without pretending unavailable providers are healthy.

Preflight should produce `READY`, `READY_WITH_WARNINGS`, `NOT_READY`, or `UNKNOWN_DEPENDENCY` states with specific reasons and remediation. Drydock readiness becomes a critical input when available; until then, Helm must consume the current publication/validation truth rather than inventing a local validator.

Preflight family	Examples
Edition	Immutable published edition exists, compatibility supported, not quarantined
Crew	Required participants invited/accepted/ready; Captain participation mode resolved
Access	Account/capability/session valid; invitation scope consistent
Media	Required Player-safe media available and authorized
Presentation	Required Lanternwake assets/fallbacks available where applicable
Drydock	Verification receipt/current readiness when project exists
Landfall	Location requirement, permission/fallback policy, map availability
Watchglass/helper	Pairing/provider readiness when required
Accessibility	Required accommodations and alternatives are available
Network/offline	Connection mode supports required Chronicle behaviors
Schedule	Planned time/timezone and Captain availability are informationally clear

26. Launch, Scheduling, and Waiting Room Coordination

Launch is an authoritative lifecycle transition. Helm should present the waiting crew, readiness changes, scheduled time, and any launch blockers in real time.

Scheduled start is not a magical timer that assumes the Captain exists. If a separate durable scheduler eventually launches under policy, Helm must reflect that authority. Otherwise schedule remains planned timing and the Captain explicitly launches.

Participating Captains enter the Player waiting room through their Player membership while retaining the ability to return to Helm. The system must avoid deadlocking launch because the Captain is simultaneously expected to be a Player.

- Double launch attempts are idempotent or conflict safely.
- Launch success appears only after canonical state commit.
- Players missing the realtime transition can refresh into the correct active state.
- The Captain may not change the pinned edition through launch; version selection is already frozen.

27. Realtime, Concurrency, and Multi-Device Operation

Captain operation frequently involves multiple tabs or devices. Helm must assume this will happen and use expected sequences, revision tokens, idempotency keys, and server-confirmed state rather than relying on one pristine browser session.

Two Captain devices issuing conflicting commands should produce one committed result and one structured stale/conflict response where appropriate. The stale UI must then refetch before presenting further actions.

Realtime is an optimization over durable truth. If realtime disconnects, polling or explicit refresh must restore the correct projection without losing command history or participant state.

Condition	Required behavior
Two devices, same command/idempotency	Return/reconcile same authoritative result
Two devices, different command, same expected sequence	One commits; the other receives stale/conflict and refreshes
Realtime gap	Detect sequence gap and refetch/replay authoritative events
Captain reconnect	Restore current Voyage projection and pending work
Browser history/back	Never revert server state; reopen current projection
Participating Captain has Player and Captain tabs	Each tab remains constrained to its projection and capability

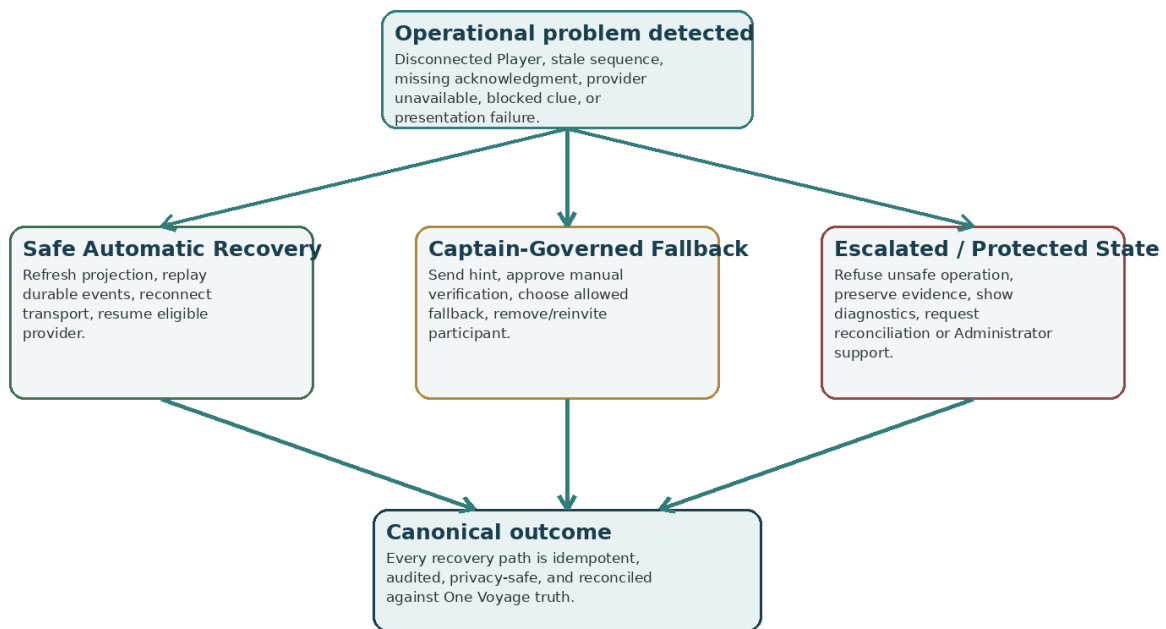
28. Recovery and Reconciliation

Helm treats recovery as a designed workflow. The Captain should not need to guess whether reloading the page will duplicate a command, lose a Player, or reveal the wrong content.

Recovery begins by diagnosing the source: transport, membership, provider, progression sequence, presentation acknowledgment, or local client state. The UI then offers only recovery actions sanctioned by the owning subsystem.

Reconciliation must be idempotent and evidence-producing. If an automatic repair cannot be proven safe, Helm escalates rather than inventing state.

Helm recovery decision flow



29. Participant Lifecycle Management

Helm should give Captains controlled participant operations without collapsing invitations, memberships, accounts, and presence into one concept.

Captains may invite, replace invitations, remove a participant when policy permits, re-invite, inspect readiness, and understand whether a Player is merely disconnected versus actually removed. Account-level suspension or deletion remains outside Helm.

For participating Captains, removing the Captain from Player membership must not remove Captain authority unless the Captain explicitly performs that separate operation and possesses permission to do so.

Action	Primary owner	Helm behavior
Invite/replace/revoke invitation	InvitationService	Use canonical lifecycle and audit
Remove Player from Voyage	Membership authority	Confirm consequence; invalidate access under policy
Reconnect Player	Realtime/session	Guide refresh/rejoin; no fake membership creation
Change crew role	Membership authority	Only when Chronicle/policy allows and history is preserved
Transfer Captain authority	Future scoped authority contract	Must be explicit, audited, and not inferred from crew role
Suspend account	Admiralty/Wayfarer	Not exposed in Helm

30. Emergency Controls and Degraded Operation

Hard Boundary
Helm may orchestrate recovery that canonical systems already support. It may not mutate the database directly, fabricate events, bypass authorization, or turn an unsupported repair into a UI shortcut.

Emergency controls are intentionally rare. Helm must not turn every difficult situation into a giant red “force it” button.

Allowed emergency controls may include pause, reconciliation request, governed manual verification, safe participant removal, or narrowly supported undo/compensation. Any stronger capability must exist in One Voyage or another owning subsystem first.

Degraded operation should remain useful. If a location, vision, notification, or presentation provider is unavailable, Helm should state what is unavailable, what is still authoritative, and which approved fallback is available.

31. Captain Intelligence and Privacy Boundaries

Captain power is operational, not omniscient. Helm must explicitly classify each data field by why the Captain needs it.

For example, a Captain may need to know that a Player submitted an incorrect answer, but not necessarily the exact text typed. A Captain may need to know that a Player has not acknowledged a reveal, but not the Player's private reflection about it.

Private Player Memories, reflections, account security data, unrelated provider identities, raw device fingerprints, and hidden personal notes stay outside Helm unless a separate explicit sharing or support contract applies.

Data class	Examples	Captain access
Operational public-to-crew	Display name, crew role, readiness	Allowed
Operational summary	Connection state, sync lag, pending request, correctness result	Allowed
Captain-authorized story detail	Captain instructions, approved evidence summary	Allowed only for authorized Voyage and current edition
Player-private	Reflection, private Memory, private note, unshared answer text	Denied by default
Creator-private	Draft notes, future unpublished content	Denied in live Helm
Security/admin	Email, sessions, provider IDs, IP, account support data	Denied; Admiralty/Wayfarer only

32. Security and Authorization

Every Helm route, projection, mutation, asset, realtime channel, and recovery operation is server-authorized against the current authenticated account and current Voyage relationship.

Role visibility in the UI is never authority. A user who manually navigates to a Captain route without resource-scoped authority receives a safe denial and no Captain projection.

Participating Captain support makes projection separation even more important: the same authenticated account can legitimately access both Player and Captain APIs, but each API must return only its contract.

- CSRF protection for cookie-authenticated mutations.
- Rate limiting for high-risk or repeatedly failing command paths.
- No raw invitation tokens, passwords, PINs, session cookies, or private story payloads in logs.
- Sequence/idempotency checks on retry-prone commands.
- Authorization revalidation on realtime reconnect and resource switch.
- Safe 404/denial for unauthorized Voyage IDs without data leak.
- Audit of privilege assignment/revocation and high-risk commands.

33. Audit, Diagnostics, and Support Boundaries

Helm operational history should be useful to a Captain, while security diagnostics and platform support remain separate.

Captain-visible audit includes Voyage configuration, launch, pause/resume, hints, reveals, verification decisions, participant changes, recovery actions, and command conflicts. It is scoped to authorized Voyages.

Platform diagnostics such as raw stack traces, database internals, account security events, deployment settings, and provider credentials belong to Admiralty or engineering observability, not Helm.

Record	Captain-visible?	Notes
Captain command receipt	Yes	Humanized action and result
Voyage event	Yes when operationally relevant	Player-safe/private payload rules still apply
Presence transition	Yes	No raw network identifiers
Security login event	No	Wayfarer/Admiralty
Provider internal error trace	No; safe summary only	Raw detail to logs/Admiralty
Reconciliation receipt	Yes safe summary	Include correlation ID and state outcome

34. Data Model and Service Contracts

Helm should minimize new persistence. Much of its value comes from projections over existing account, session, membership, event, presence, verification, and audit data.

New persistence is justified only for facts that do not already have an authoritative home, such as optional Captain authority history if current scoped roles cannot represent it, saved Captain dashboard preferences, explicit preflight receipts, or recovery case records.

Do not create a “HelmState” JSON blob that becomes a shadow copy of the Voyage.

Concept	Preferred source
Captain authority	Existing scoped role/resource assignment or minimal dedicated assignment if required
Player participation	PlaythroughMembership
Voyage lifecycle	TaleSession
Progression	TaleSessionEvent / canonical progression state
Presence	Existing presence evidence/realtime service
Command availability	Derived from canonical state + policy
Needs Attention	Derived/cached projection with source references
Preflight receipt	Versioned derived receipt tied to session/version and provider evidence
Recovery action	Canonical command/event plus optional recovery receipt
Captain UI preferences	User preference namespace, not Voyage truth

35. API and Projection Contracts

Helm APIs should return explicit view models, never raw ORM entities. The UI should receive enough information to render truthfully without receiving fields it should merely ignore.

Recommended projections include CaptainLibraryProjection, CaptainVoyageProjection, CaptainCrewMemberProjection, CaptainProgressProjection, CaptainNeedsAttentionItem, CaptainCommandOption, CaptainPreflightReceipt, and CaptainRecoveryProjection.

Mutations should remain narrow canonical actions rather than a generic “patch Voyage” endpoint.

Capability	Representative contract
Captain library	GET /api/captain/library
Voyage console	GET /api/captain/voyages/:id
Crew projection	GET /api/captain/voyages/:id/crew
Preflight	POST or GET governed preflight endpoint with exact-source receipt
Command preview	POST /api/captain/voyages/:id/commands/preview
Command execute	POST /api/captain/voyages/:id/commands
Recovery	POST /api/captain/voyages/:id/recovery/:action
Perspective switch	Navigation only; Player endpoint remains ordinary Player API

36. Responsive Captain Experience

Captain operating mode must be practical while the Captain is physically doing something else. Desktop may show crew, progress, event stream, and action rail together. Tablet should preserve a two-pane command layout. Phone should prioritize Needs Attention, current status, crew, and a bottom-sheet action/recovery system.

Critical actions require large touch targets and clear confirmation. Hover-only information is prohibited. Long event history and detailed graph inspection may collapse behind tabs or sheets on small screens, but current state and urgent warnings remain visible.

Second-monitor mode should support dense-but-readable information without requiring fullscreen cinematic behavior. Helm is an operating surface, not a trailer.

37. Accessibility

- All status has text equivalents; color, motion, sound, and vibration are supplementary only.
- Keyboard users can reach every operational action, warning, participant, filter, and recovery control in logical order.
- Dialogs trap focus and return it to the triggering control after confirmation or cancellation.
- Live regions announce meaningful changes such as Player ready, disconnection, Captain command result, or launch without becoming a machine-gun notification feed.
- At 200% zoom, critical status and actions remain usable without horizontal page scrolling except intentionally scrollable graph regions.
- Phone controls meet touch target requirements and do not place dangerous actions adjacent without separation.
- Reduced-motion mode preserves command confirmation, state change, and attention hierarchy without large travel or continuous effects.
- Charts/progress maps provide structured text alternatives for current step, completed steps, pending requirements, and warnings.

38. Lanternwake Motion and Presentation

Helm uses Lanternwake for meaningful Captain presentation: Voyage launch confirmation, major reveal confirmation, critical recovery completion, and perhaps a restrained Captain workspace arrival. Ordinary operational updates should favor fast interface motion rather than ceremony.

Animations begin after authoritative state commits where success is implied. Pending commands have a distinct visual state; failure restores or reconciles controls rather than animating success.

Participating Captain perspective switching should be restrained and fast. The Player side retains the Player cinematic system and must not inherit Captain dashboard motion ownership.

Moment	Recommended presentation
Captain workspace arrival	Short contextual transition
Launch committed	Meaningful ceremony with immediate truthful fallback
Player becomes ready	Subtle list/status transition
High-risk command success	Clear confirmation, not a giant celebration
Command failure	No success motion; concise error and recovery
Critical reconciliation complete	Stable recovery confirmation
Realtime reconnect	Small state restoration indicator

39. Integration with Adjacent Programs

Helm must be designed so adjacent programs can deepen it without forcing a rewrite.

Drydock will eventually supply stronger launch-readiness evidence. Landfall and Watchglass will supply provider-specific verification state. Wakebook will present participating-Captain history. Admiralty will consume safe operational summaries and escalation hooks. Shipwright may expose authoring metadata that Helm reads from the immutable published edition, never the draft.

Program	Future Helm integration
Drydock	Preflight readiness, validation issues safe for Captain, scenario warnings
Landfall	Live location-provider status, arrival/manual fallback eligibility
Watchglass	Vision verification status, certification, safe evidence summary
Wakebook	Captain-as-Player history and intervention summaries
Tideglass	Edition changes before selecting a version/replay
Admiralty	Escalated support/repair and platform-health links
Shipwright	Published Captain instructions and authored operational metadata only

40. Performance and Operational Architecture

Helm is a live dashboard, so expensive full projections on every heartbeat are unacceptable. Separate high-frequency presence/sync updates from lower-frequency Chronicle graph or history data.

The client should reconcile by event sequence/version and update only affected regions. Event history and completed records must paginate. Large graphs should virtualize or collapse inactive sections. Provider diagnostics load on demand.

Operational targets should be recorded during implementation, but the initial UX goal is that command availability and urgent crew state feel immediate under ordinary local/network conditions while remaining truthful when stale.

- No polling interval may create unnecessary database load merely to animate “live” status.
- Prefer authenticated realtime with bounded polling fallback.
- Presence payloads stay small and privacy-safe.
- Captain Library summary queries should avoid N+1 per Voyage.
- Graph and event history use pagination/virtualization where needed.
- Mobile low-power mode may reduce ambient updates without hiding state.

41. Testing and Acceptance Matrix

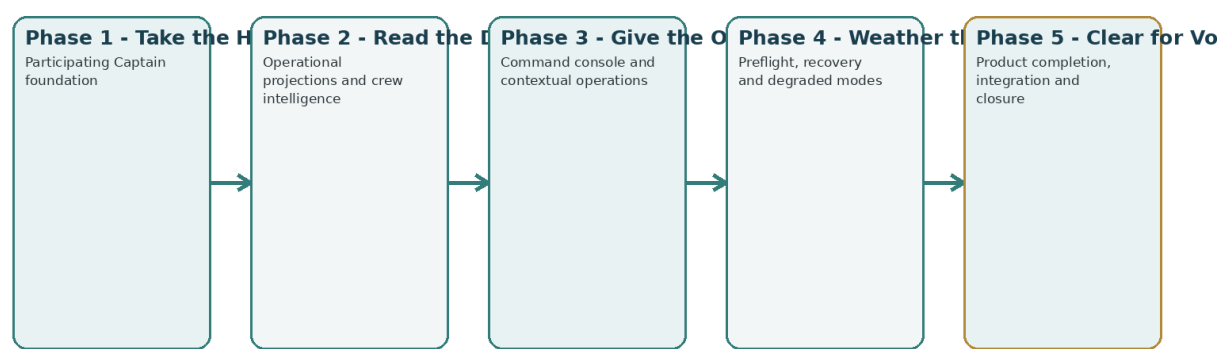
Test family	Required evidence
Authority separation	Captain-only, Player-only, Captain+Player, unauthorized account
Projection privacy	Player API never receives Captain fields; Captain projection never receives prohibited Player/private account fields
Participation lifecycle	Create participating Captain, late join, remove Player membership, revoke Captain authority independently
Commands	Preview, stage, execute, idempotency, stale sequence, multi-device conflict, high-risk confirmation
Presence/realtime	Connect, lag, disconnect, reconnect, missed event, polling fallback
Preflight	Ready, warnings, hard blocker, unknown provider, later Drydock receipt
Recovery	Reconnect, reconciliation, provider fallback, presentation acknowledgment issue, participant removal/reinvite
History	Captain interventions and Player history remain source-correct and version-pinned
Responsive	Desktop, tablet, phone, 200% zoom, long names, large crews
Accessibility	Keyboard, focus, live regions, Axe, reduced motion, screen-reader structure
Security	IDOR, direct URL, replayed mutation, CSRF, revoked authority, stale session
Regression	Existing FT-007, invitations, Player Library, Creator separation remain functional

42. Implementation Phases Overview

Project Helm is divided into five phases. Each phase is intentionally designed as a mainline-safe plateau. No phase may depend on an unmerged later Helm phase for current correctness.

The phases follow the new Voyagewright continuous-development doctrine: branch from current main, implement, reconcile against any intervening mainline changes, pass Sounding Line, merge, then start the next Helm phase from the new integrated main.

Project Helm phased mainline architecture



Each phase: branch from current main -> implement -> reconcile -> Sounding Line -> merge -> next phase starts from new main

No Helm phase stack is allowed to age beside main. Incomplete project is fine; incomplete behavior is not.

Phase	Name	Primary outcome	Integration pattern
1	Take the Helm	Captain authority and Player participation become independent; participating Captain works end-to-end	Compatible migration / vertical slice
2	Read the Deck	Deep operational projections, crew state, presence, warnings, event stream	Additive foundation + vertical slice
3	Give the Orders	Full contextual Captain console, command lifecycle, hints, approvals, progress map	Vertical slice
4	Weather the Passage	Preflight, recovery, provider fallback, degraded operation, participant repair	Vertical slice / recovery foundation
5	Clear for Voyage	Responsive/accessibility/motion completion, adjacent integrations, owner walkthrough, closure	Product completion / retirement

43. Phase 1 - Take the Helm

Phase 1 establishes the architectural change that motivates the project: Captain authority and Player participation become independent, first-class relationships.

It must support Captain-only and Captain+Player setup using the current canonical account/session system. A participating Captain receives an ordinary Player membership, ordinary Player-safe APIs, normal artifact/history participation, and ordinary crew presence while retaining separate resource-scoped Captain authority.

Phase 1 should avoid a large Captain UI rebuild. It may add the participation selector to Voyage setup, perspective switching, minimal Captain projection needed to prove the boundary, and focused tests.

Mainline Integration Field	Phase 1 Contract
Starting prerequisites	Current canonical account/session, One Voyage membership/progression, FT-007 Captain operations in main
Phase ownership	Captain participation mode and authority-vs-membership contract
Activation state	Active vertical slice
Backward compatibility	Existing Captain-only Voyages continue unchanged
Schema impact	Prefer existing scoped roles + membership; additive schema only if current model cannot safely represent authority history
Hard dependencies	None beyond current mainline platform
Soft dependencies	Wakebook later consumes Captain-as-Player history; Drydock not required
Permanent-stop test	If Phase 2 never happens, Captains can still choose to play or only operate without breaking existing Captain workflows
Acceptance boundary	Authority/privacy tests, Player membership/artifact/history tests, responsive perspective switch, Sounding Line mainline gate

44. Phase 2 - Read the Deck

Phase 2 makes the Captain experience informative. It creates explicit operational projections rather than asking the UI to assemble truth from unrelated raw endpoints.

Deliver Captain Library prioritization, crew operational state, presence/sync health, Needs Attention, event stream, and a first live progress projection. No new progression mutations are invented.

Projection contracts must be privacy-reviewed before broad UI work because this phase is where the temptation to “just send everything to the Captain client” becomes strongest.

Mainline Integration Field	Phase 2 Contract
Starting prerequisites	Helm Phase 1 accepted in main
Phase ownership	Captain operational projection types, warning taxonomy, crew status presentation
Activation state	Active additive upgrade
Backward compatibility	Existing Captain command endpoints remain valid
Schema impact	Prefer derived projections; optional bounded warning/cache persistence only if justified
Hard dependencies	Helm P1
Soft dependencies	Deepwater findings may refine projections
Permanent-stop test	Captain Library/Console are substantially more informative even without Phase 3 action redesign
Acceptance boundary	Projection privacy, presence/realtime, large crew, stale data, responsive UI, Sounding Line gate

45. Phase 3 - Give the Orders

Phase 3 turns the informational console into a deep operating surface by wrapping canonical Captain commands in contextual intent, prerequisites, previews, confirmations, receipts, and clear outcomes.

Deliver the progress map, contextual action rail, command families, hint workflow, approval/verification controls, safe Player preview, and command conflict handling. Existing backend command types remain the authority; user-facing language may be friendlier.

The phase must prove that commands work from multiple Captain devices, stale projections fail safely, and no success presentation occurs before the server commits the result.

Mainline Integration Field	Phase 3 Contract
Starting prerequisites	Helm P2 in main; current command service and One Voyage progression healthy
Phase ownership	Captain command UX and operational command projection
Activation state	Active vertical slice
Backward compatibility	Existing API behavior remains; old thin controls may delegate until retirement
Schema impact	Minimal; command/audit source remains One Voyage/platform
Hard dependencies	Helm P1-P2
Soft dependencies	Drydock/Watchglass/Landfall enhance later command contexts
Permanent-stop test	Helm is already a capable live command center even without advanced recovery/preflight
Acceptance boundary	Command preview/execute, stale/idempotent/multi-device, privacy, browser and owner-facing UX evidence

46. Phase 4 - Weather the Passage

Phase 4 makes Helm resilient under imperfect real-world operation. It adds comprehensive preflight, recovery, reconciliation, provider-degraded states, participant repair, and safe emergency boundaries.

Where adjacent projects are not yet present, Helm must use provider-neutral contracts and truthful unavailable states. It must not fake a Drydock, Landfall, or Watchglass result simply to make preflight green.

This phase should make “something went wrong during the Voyage” a supported product state rather than a request to reload and hope.

Mainline Integration Field	Phase 4 Contract
Starting prerequisites	Helm P3 in main
Phase ownership	Preflight orchestration, recovery UX, degraded-mode decisions, repair receipts
Activation state	Active; provider-specific integrations gated by availability
Backward compatibility	Normal healthy Voyage path unchanged
Schema impact	Versioned preflight/recovery receipts only if persistence is necessary
Hard dependencies	Helm P1-P3
Soft dependencies	Drydock, Landfall, Watchglass provider contracts when available
Permanent-stop test	Healthy and degraded Voyages remain operable; missing future providers show truthful unavailable/fallback states
Acceptance boundary	Fault injection, restart/reconnect, provider unavailable, recovery idempotency, security and accessibility gates

47. Phase 5 - Clear for Voyage

Phase 5 closes the project as a product rather than merely as architecture. It completes responsive behavior, accessibility, Lanternwake integration, performance, documentation, historical hooks, and visual/behavioral polish.

It also retires or redirects any remaining competing thin Captain surfaces only after their capabilities are proven through Helm. Creator authoring shortcuts remain navigation links, not embedded controls.

The final gate requires a real owner walkthrough of Captain-only and Captain+Player Voyages, including launch, active operation, Player perspective, command conflicts, disconnection, recovery, completion, and return to historical views.

Mainline Integration Field	Phase 5 Contract
Starting prerequisites	Helm P4 in main; adjacent provider integrations consumed only where their contracts are mainline
Phase ownership	Final Captain product experience and Helm-specific compatibility retirement
Activation state	Fully active
Backward compatibility	Only proven obsolete Captain compatibility surfaces may retire
Schema impact	No speculative new domain; cleanup only when safe
Hard dependencies	Helm P1-P4
Soft dependencies	Wakebook/Tideglass/Admiralty links may deepen later without reopening Helm
Permanent-stop test	Project Helm remains complete and useful even if no future project integrations are built
Acceptance boundary	Full Sounding Line gate, owner walkthrough, route/navigation reachability, accessibility, responsive, privacy, recovery and completion evidence

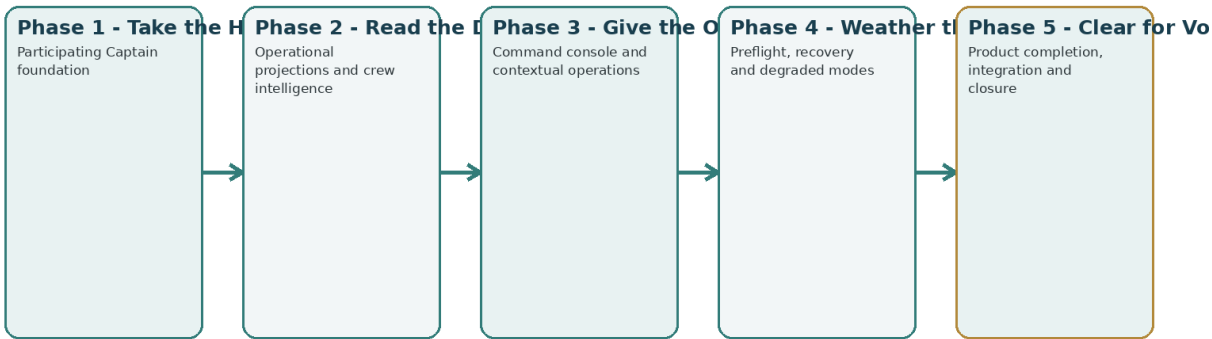
48. Mainline Integration Architecture

Project Helm is governed by the Voyagewright Continuous Development and Mainline Integration Standard. Multiple projects may run alongside Helm, but Helm may not build a stack of unmerged phases.

Each Helm phase begins from current accepted origin/main. When main advances during the phase, the Helm branch reviews and reconciles intervening changes before acceptance. Sounding Line determines which evidence is invalidated.

Helm has moderate schema pressure around membership and Voyage authority. Concurrent projects, especially Wakebook, must not independently redefine membership semantics. Helm owns runtime participation truth; Wakebook consumes that truth for historical presentation.

Project Helm phased mainline architecture



Each phase: branch from current main -> implement -> reconcile -> Sounding Line -> merge -> next phase starts from

No Helm phase stack is allowed to age beside main. Incomplete project is fine; incomplete behavior is not.

Concurrency relationship	Rule
Helm P1 + Wakebook P1	May run in parallel if Wakebook treats membership semantics as read-only and derived
Helm + Admiralty	Parallel; Admiralty consumes admin/observability seams, not Helm command ownership
Helm + Drydock	Parallel; Drydock owns authoring validation, Helm owns live operation
Helm + Shipwright	Parallel if Shipwright stays in Studio UX; no shared live Captain UI files
Helm + Deepwater	Deepwater may audit Helm; remediation touching Helm waits/coordinates
Helm + Tideglass	Mostly independent
Helm phase N+1	May not start until Helm phase N is accepted in main

49. Risks and Failure Modes

Risk	Why it matters	Required mitigation
Privilege leakage into Player view	Participating Captain uses same account for both roles	Strict projection/API separation and hostile tests
Duplicate authority model	Helm invents parallel progression or membership truth	Canonical ownership matrix and code review
Dashboard overexposure	Raw backend data sent because Captain is trusted	Purpose-limited projections and privacy matrix
Command button explosion	Every command visible all the time	Contextual command availability and progressive disclosure
Stale multi-device state	Two Captain devices act on old sequence	Expected sequence, idempotency, conflict refresh
Recovery becomes direct repair	UI bypasses normal services	Only canonical recovery ports and audited receipts
Captain+Player launch deadlock	Captain counted as Player but operating launch	Explicit readiness rules and perspective-independent launch authority
Provider optimism	Unavailable Landfall/Watchglass shown healthy	Truthful unavailable/degraded states
Mobile unusability	Captain must operate away from desktop	Phone-first operational acceptance
Phase merge debt	Multiple Helm phases stack while other projects change membership	Mandatory phase-level convergence

50. Final Acceptance Criteria

- A Captain can create and operate a Voyage using the canonical account/session and canonical One Voyage progression system.
- A Captain can choose Captain-only or Captain+Player mode where the Chronicle/policy permits.
- A participating Captain receives a normal Player membership and cannot see Captain-only data through Player APIs or UI.
- Revoking Captain authority does not silently remove Player membership, and removing Player membership does not silently revoke Captain authority.
- Captain Library clearly prioritizes Needs Attention, Active, and Ready to Launch Voyages.
- Live Helm shows truthful crew readiness, presence, synchronization, pending requests, progress, warnings, and event history.
- Every Captain command is context-valid, server-authorized, idempotent/retry-safe where required, conflict-safe, and audit-linked.
- High-risk commands use explicit human confirmation and do not animate success before committed truth.
- Preflight reports exact blockers/warnings and consumes external provider/Drydock evidence only when actually available.
- Recovery handles disconnect, stale state, missed events, provider failure, participant loss, and presentation acknowledgment problems through governed paths.
- Captain operational intelligence excludes prohibited Player-private, Creator-private, security, and admin data.
- The complete Captain experience works on desktop, tablet, phone, keyboard-only, 200% zoom, reduced motion, and screen-reader-oriented structure.
- No user-facing Captain route is orphaned; ordinary Captain destinations are reachable through visible product navigation.
- All five phases entered main independently under Sounding Line evidence and no same-project phase stack was required.
- Final major product acceptance is withheld until owner walkthrough succeeds.

51. Governance and Change Control

This document is the governing baseline for Project Helm. Later implementation records may refine names, file paths, or provider contracts as the repository evolves, but they may not silently weaken the central invariants.

Changes requiring a formal governance amendment include collapsing Captain authority and Player membership back into one relationship; allowing Captain privilege to alter Player-safe projections; creating a second progression engine; moving platform administration into Helm; or permitting live authoring mutations from Captain operation surfaces.

Ordinary implementation changes such as component names, route syntax, projection field names, or storage mechanisms may adapt to current repository conventions when the product contracts remain intact.

Every phase completion record must state the actual mainline SHA, Mainline Safety Contract result, activation state, migrations, compatibility retained/retired, Sounding Line decision, and owner-acceptance status.

52. Glossary

Term	Meaning
Captain Authority	Server-enforced right to operate a specific Voyage or governed set of Voyages
Player Membership	Normal participation relationship that grants Player-safe story access and personal progression/history
Participating Captain	One account that simultaneously has Captain authority and a normal Player membership for the same Voyage
Operational Projection	Purpose-limited Captain view derived from canonical state
Needs Attention	Prioritized queue of conditions where Captain action may materially help
Command Preview	Nonmutating evaluation of prerequisites and projected effect
Staged Command	Persisted/prepared command intent when current command architecture supports staging
Command Receipt	Canonical committed or staged result with truth about persistence/publication
Preflight Receipt	Versioned readiness result tied to exact Voyage/version/provider evidence
Recovery Action	Governed attempt to restore operability through owning subsystem contracts
Player Perspective	Ordinary Player-safe experience for a participating Captain
Safe Player Preview	Nonmutating synthetic Player view available to a Captain who is not a Player
Operational Omniscience	Prohibited anti-pattern where Captain receives data merely because the database has it

53. References

Primary repository basis at preparation: Kgray44/treasurehuntSoT main at f1c2f22dd935322c1a71eb80c51592f243dc196d.

Forever Treasure Feature Catalog, especially FT-001 Unified Chronicle Platform, FT-002 Secure Account/Session/Role System, FT-005 Invitation and Crew Lifecycle, FT-007 Captain Voyage Operations, FT-014 Authoritative Progression Engine, FT-021 Realtime Delivery/Presence/Replay, FT-023 Verification Provider Framework, and FT-026 Universal Language and Chronicle Design System.

Tall Tale Platform Codex Implementation Specification, especially Captain Library and Command Workspace, invitation workflow, authorization, versioning, state machines, service boundaries, realtime truth, responsive Captain operating mode, recovery, audit history, and testing strategy.

Current source references include src/domain/admin.ts and src/server/admin-command.ts for command taxonomy, risk, presence, preview/staging, idempotency, sequence conflicts, audit receipts, and canonical command delegation.

Voyagewright Continuous Development and Mainline Integration Standard for phase-level mainline safety, dependency classes, branch lifecycle, migration ownership, and prompt convergence.

Project Sounding Line governing documents for verification tiers, contract ownership, evidence, mainline acceptance, browser/accessibility validation, and release decisions.

Appendix A - Captain Command Catalog

The following catalog describes how Helm should group the current canonical command vocabulary. The exact command registry remains source-controlled by the owning runtime; Helm must revalidate current code before each implementation phase.

Command	Intent	Risk family / UI expectation
PREPARE_CHAPTER	Prepare next chapter transition	Medium - preview inline
RELEASE_CHAPTER	Release chapter to Players	High - confirm and receipt
MARK_SOLVED	Mark current problem solved	High
COMPLETE_CHAPTER	Complete current chapter	High
PREPARE_HINT	Stage an allowed hint	Medium
RELEASE_HINT	Release selected hint	High
RELEASE_NEXT_HINT	Release next governed hint	High
REVEAL_MAP / REVEAL_ROUTE	Reveal geography	High
AWARD_ARTIFACT	Grant artifact through canonical rules	High
REVEAL_ARTIFACT_SILHOUETTE	Reveal artifact state	High
CONNECT_ARTIFACTS	Change artifact relationship state	High
DISCOVER/UPDATE/ADVANCE/ COMPLETE_SIDE_QUEST	Operate optional branch	High
ADD_JOURNAL_ANNOTATION / ADD_LOG_ENTRY	Add governed narrative/operational entry	High
RELEASE_JOURNAL_ENTRY	Release authored runtime entry	High
TEASE_FINALE / UPDATE_FINALE_REQUIREMENT	Operate finale readiness	High
PAUSE / RESUME	Voyage lifecycle control	High
UNDO_LAST	Compensating/reversal path when supported	High / recovery-only
REQUEST_RECONCILIATION	Request safe state reconciliation	Low / recovery

Appendix B - Operational Projection Contract

Projection	Required fields / behavior
CaptainLibraryProjection	Grouped Voyages, safe Chronicle identity, edition, status, attention summary, last activity, next action
CaptainVoyageProjection	Lifecycle, current progression, sequence, crew summary, provider summary, warnings, command options
CaptainCrewMemberProjection	Safe identity, membership, role, presence, sync lag, pending request, readiness
CaptainNeedsAttentionItem	Stable ID, severity, source, created time, message, recommended action, blocking status
CaptainCommandOption	Intent, canonical command ID, target, risk, prerequisites, enabled/disabled reason, preview availability
CaptainPreflightReceipt	Exact version/session identity, readiness result, checks, provider evidence refs, blockers/warnings, generated time
CaptainRecoveryProjection	Detected issue, source facts, available safe recovery actions, escalation requirement
CaptainEventItem	Sequence/timestamp, safe actor, category, human label, safe metadata, correlation link

Appendix C - Captain Privacy Matrix

Information	Player view	Captain view	Admiralty/support
Player display name / crew role	Yes per membership	Yes	Yes
Presence and sync state	Own/crew-safe where designed	Operational summary	Yes
Exact typed answer text	Own submission if UI shows it	No by default; only if Chronicle explicitly authorizes	Only under support/privacy contract
Correct/incorrect result	Own	Yes when operationally relevant	Yes
Private reflection / Memory	Owner only	No	Only with explicit support access if ever governed
Captain instructions	No	Yes for authorized edition	Yes where support scope permits
Future unpublished draft	No	No	Creator/Admin only under governing authority
Email / linked provider ID	Own account settings	No	Wayfarer/Admiralty only
IP / device fingerprint	No	No raw data	Security/admin only
Invitation raw token	Never after creation	Never after creation	Never; hash/prefix only
Verification evidence raw media	Player/provider policy	Only if explicit verification policy permits	Protected support path only

Appendix D - Mainline Safety Matrix

Phase	Mainline-safe capability if project stops here
P1 Take the Helm	Captain-only and Captain+Player participation work safely with independent authority and membership; old Captain-only behavior still works.
P2 Read the Deck	Captain Library and Console gain reliable operational projections, crew state, warnings, presence, and history without needing P3.
P3 Give the Orders	Captain can operate deeply through contextual canonical commands; preflight/recovery remain simpler but current operations are coherent.
P4 Weather the Passage	Captain can launch with governed preflight and recover common degraded states; missing future providers fail truthfully.
P5 Clear for Voyage	Full Project Helm experience complete, polished, accessible, integrated, and owner-accepted.

Appendix E - Final Phase Acceptance Checklist

Final Governing Rule
The Captain may hold the wheel and still be part of the adventure. Voyagewright must model those facts separately, present them safely, and never make operational power an excuse to leak private story truth.

- Fresh origin/main fetched and actual phase base recorded.
- Current Continuous Development/Mainline Integration Standard reviewed.
- Phase dependencies and concurrency class recorded.
- Mainline Safety Contract written before broad implementation.
- Prisma/SQLite/MySQL impact declared and migration reservation obtained if needed.
- Captain/Player projection boundaries explicitly tested.
- No direct progression logic introduced outside One Voyage.
- Focused unit/service/component/browser/security/accessibility evidence complete.
- Current origin/main changes since phase base inspected and reconciled semantically.
- Sounding Line decides valid evidence and reruns invalidated scopes.
- Phase merged to current main without beginning the next Helm phase on the old branch.
- Post-merge integrated SHA validated as required.
- Local/remote main parity proven.
- Feature Catalog and user/developer docs updated to implemented truth.
- Owner walkthrough performed for major user-facing completion phases before PRODUCT ACCEPTED.
- Next Helm phase, if any, starts from the new accepted mainline only.